Rethinking Cluster State Management for Lightweight Function as a Service Orchestration

Prof. Ana Klimovic

ETH zürich

A story about:

- How a popular, out-of-the-box system like Kubernetes may persist more data than you really need for your use-case, limiting performance

- How we designed a clean-slate cluster manager that relaxes some state persistence to achieve 1250x higher container creation throughput, meeting the needs of serverless applications

Serverless computing paradigm

From renting virtual machines

Instance Size	vCPU	Memory (GiB)	Instance Storage (GB)	Network Bandwidth (Gbps)***
m6g.medium	1	4	EBS-Only	Up to 10
m6g.large	2	8	EBS-Only	Up to 10
m6g.xlarge	4	16	EBS-Only	Up to 10
m6g.2xlarge	8	32	EBS-Only	Up to 10
m6g.4xlarge	16	64	EBS-Only	Up to 10
m6g.8xlarge	32	128	EBS-Only	12
m6g.12xlarge	48	192	EBS-Only	20
m6g.16xlarge	64	256	EBS-Only	25
m6g.metal	64	256	EBS-Only	25
m6gd.medium	1	4	1 x 59 NVMe SSD	Up to 10
m6gd.large	2	8	1 x 118 NVMe SSD	Up to 10
m6gd.xlarge	4	16	1 x 237 NVMe SSD	Up to 10
m6gd.2xlarge	8	32	1 x 474 NVMe SSD	Up to 10
m6gd.4xlarge	16	64	1 x 950 NVMe SSD	Up to 10
m6gd.8xlarge	32	128	1 x 1900 NVMe SSD	12

Serverless computing paradigm

From renting virtual machines to using ***elastic compute & storage services***

AWS Lambda

Amazon S3

Azure Machine Learning

Google BigQuery

Serverless computing paradigm

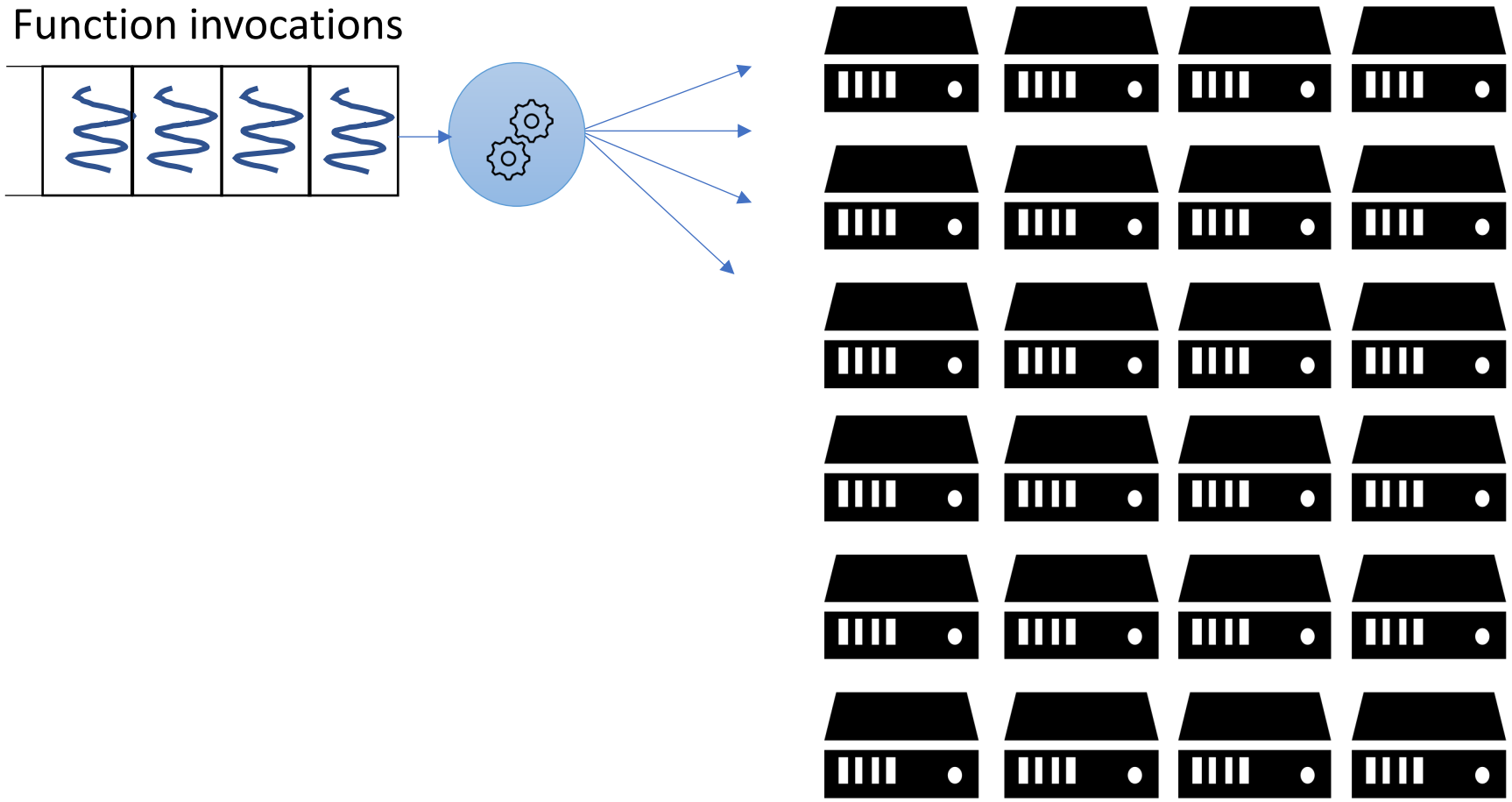
From renting virtual machines to using ***elastic compute & storage services***

Makes the cloud easier to use

- Users no longer explicitly provision and scale CPU/memory/storage resources
- Under the hood, cloud services auto-scale based on load
- Users pay for the resources consumed (pay-what-you-use)

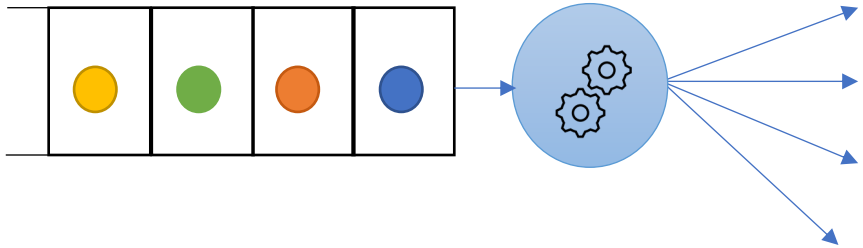
Example: **Function as a Service (FaaS)** platforms

FaaS cluster orchestration

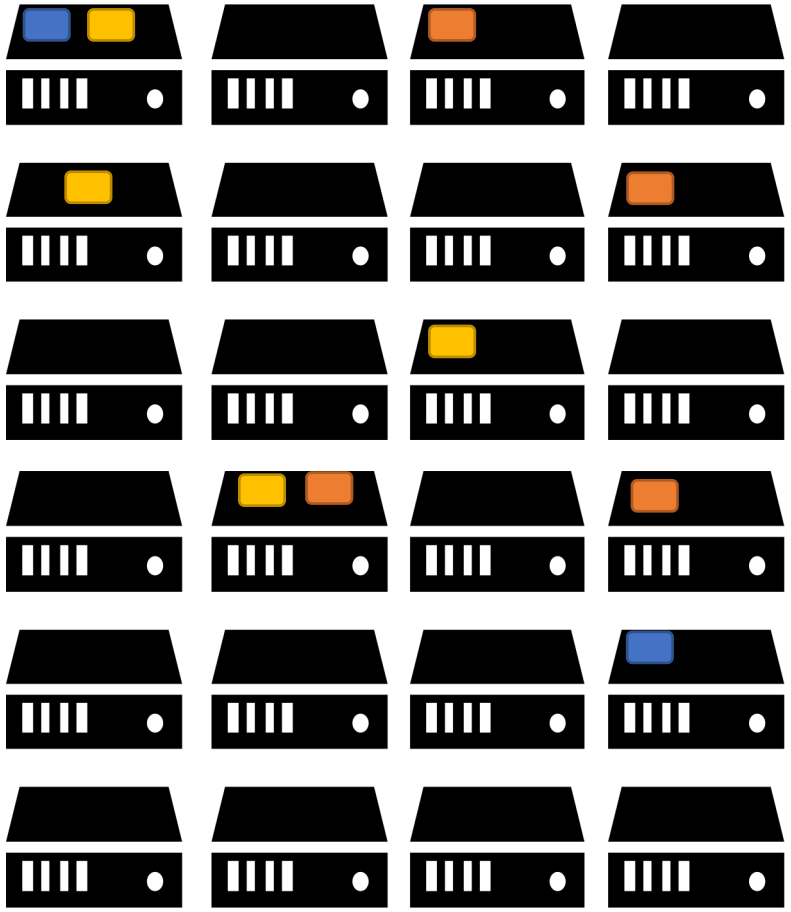


FaaS cluster orchestration

Function invocations



Pre-warmed sandboxes



Challenges:

- High request rate
- High churn

Cluster manager needs to:

- Load-balance requests
- Autoscale sandboxes
- Place sandboxes on nodes
- Provide fault-tolerance

Current approach:

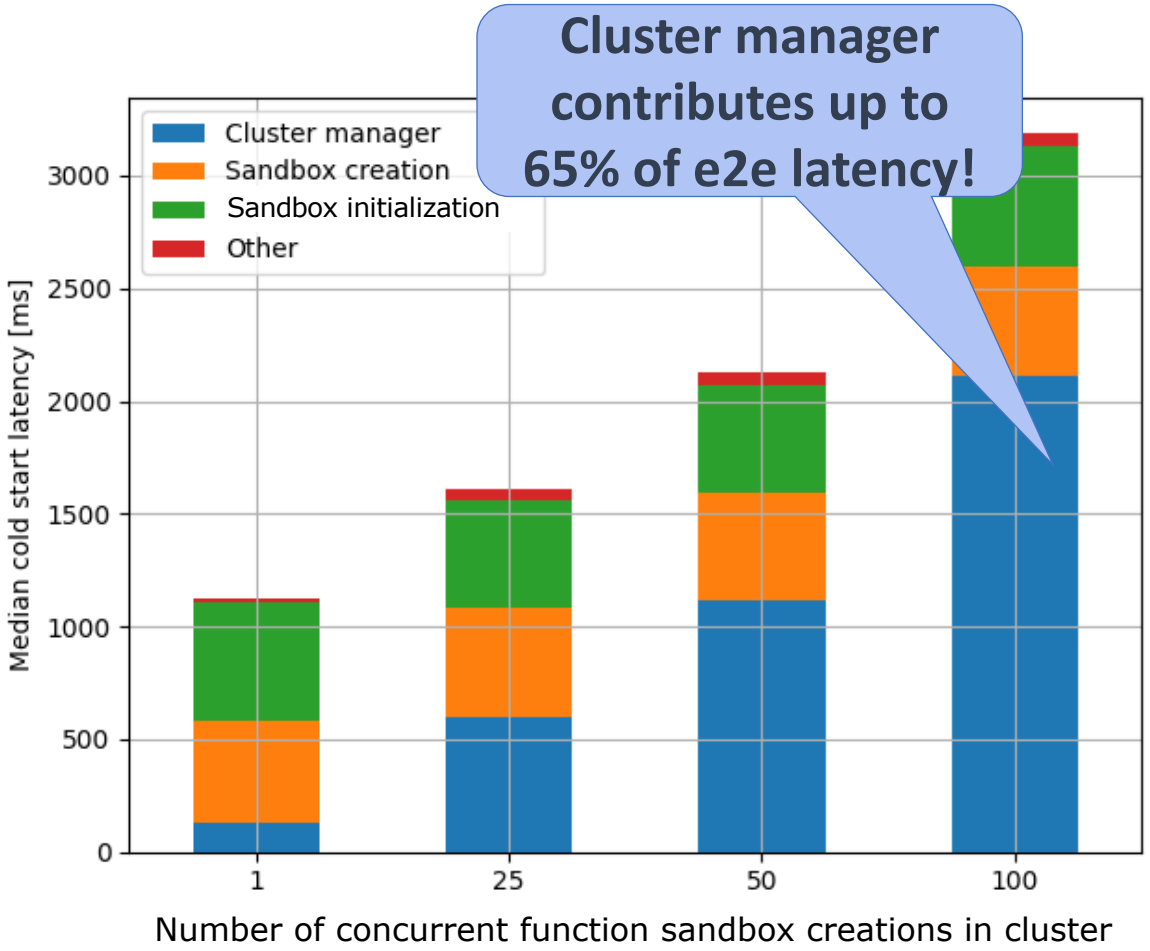
Knative

kubernetes

What is the cost of FaaS cluster orchestration?

Experiment on 100-node cluster

- Knative-on-K8s cluster manager
  - Open-source + commercially used

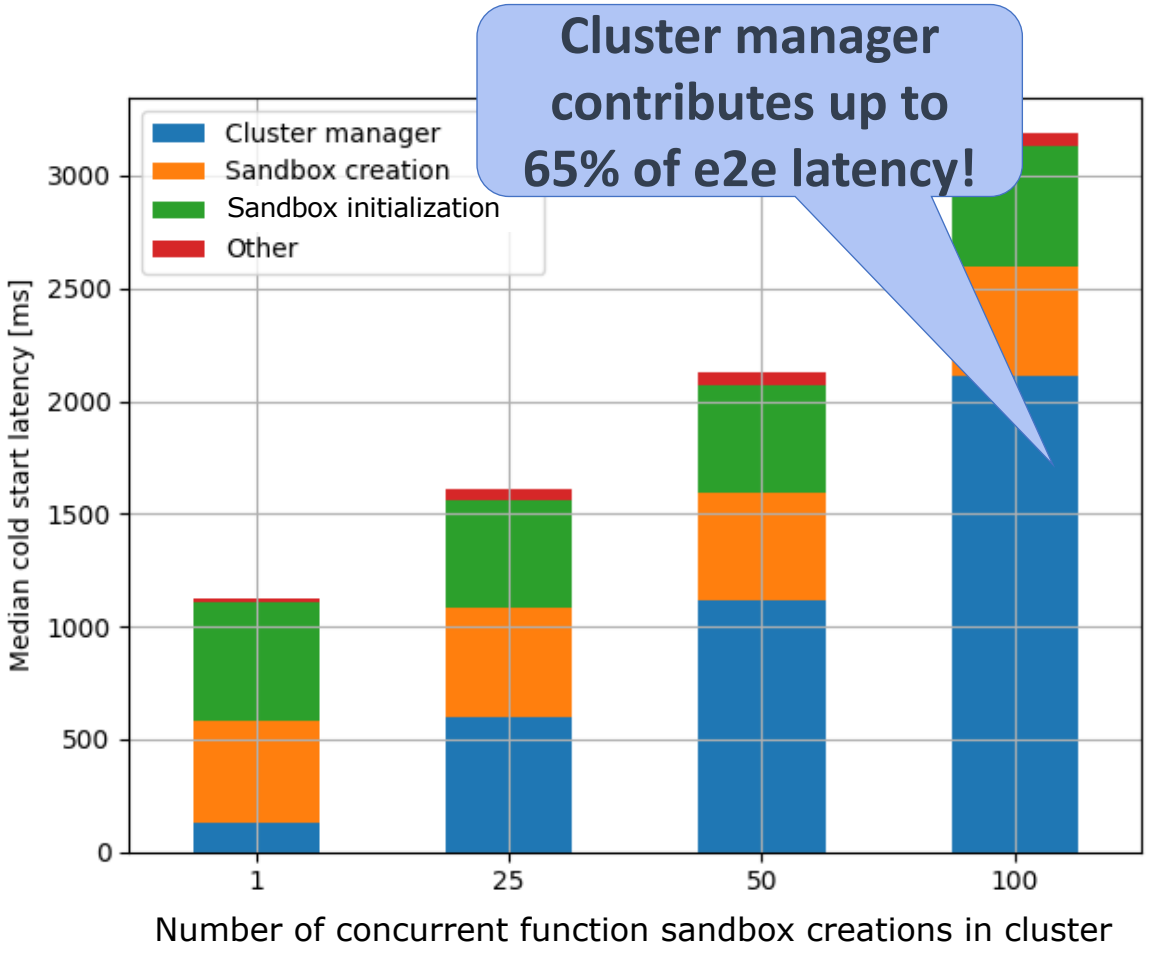


Understanding the Neglected Cost of Serverless Cluster Management. Lazar Cvetković, Rodrigo Fonseca, Ana Klimovic. *WORDS'23 at SOSP*. 2023.

What is the cost of FaaS cluster orchestration?

Experiment on 100-node cluster

- Knative-on-K8s cluster manager
  - Open-source + commercially used



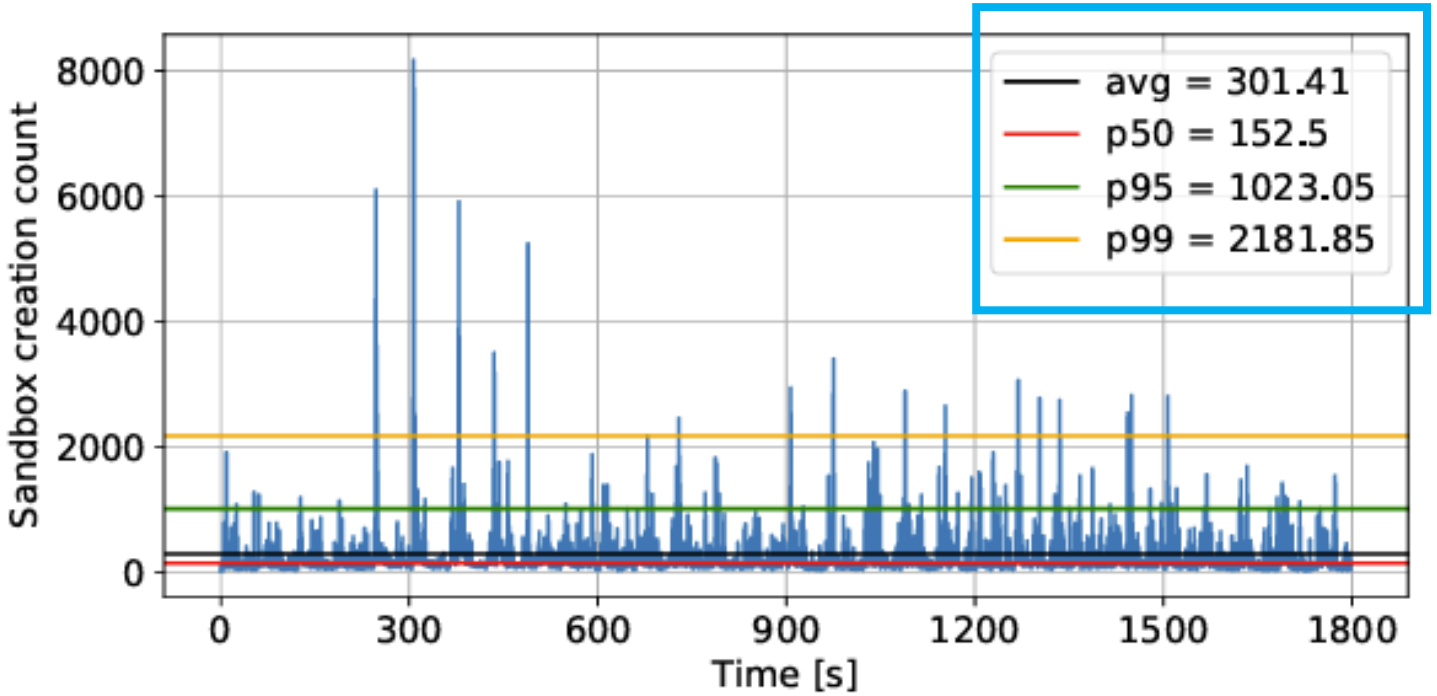
How common are sandbox creations?

- **~300 *per second*** in Azure Functions trace
- **52%** of functions experience 100% cold starts

Shahrad et al., *Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider*, ATC'20

How often are sandboxes created?

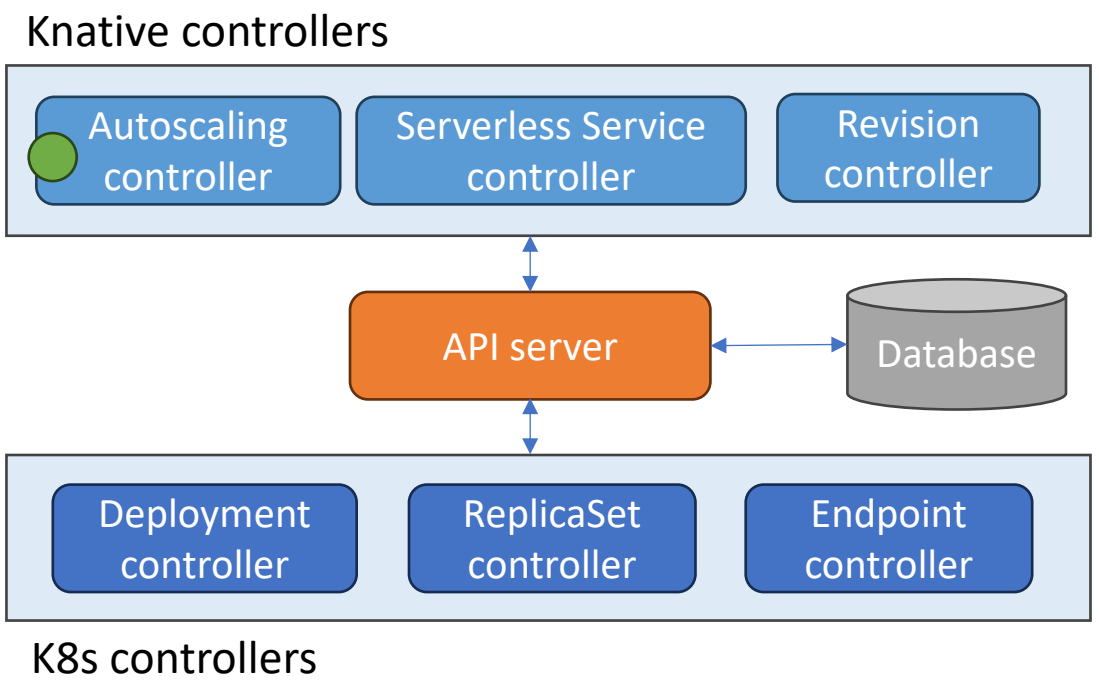
- 30-minute Azure trace simulation
- 1000-node cluster
- Default Knative-on-K8s scheduling policies
- 10-minute warmup



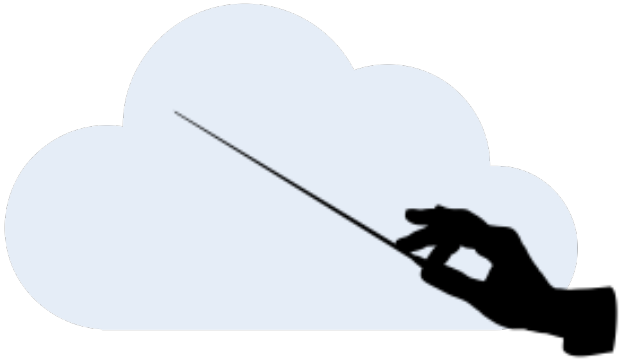
Real workload exhibits a way higher cold start rate than the state-of-the-art FaaS cluster managers can support

Where does cluster manager overhead come from?

- Complex critical path involving many controllers & control loops
  - Each function is associated with K8s Deployment, ReplicaSet, Endpoint, …
- Controllers communicate via centralized, strongly consistent DB
- High serialization overhead



Bottleneck comes from a complex system architecture and strongly consistent management of a large volume of state.



Dirigent

Lazar Cvetković

François Costa

Michal Friedman

Mihajlo Djokic

Dirigent: Lightweight Serverless Orchestration. Lazar Cvetković, François Costa, Mihajlo Djokic, Michal Friedman, Ana Klimovic. *2024*.

Dirigent: a new cluster manager for FaaS

Key design principles:

- Simplify cluster management abstractions

Abstraction	Associated State
Function	Name Image URL Port to expose Scheduling configuration Scheduling metrics
Sandbox	Name IP address Port on worker node Worker node ID
DataPlane	IP address Port
WorkerNode	Name IP address Port

Dirigent: a new cluster manager for FaaS

Key design principles:

- Simplify cluster management abstractions
- Eliminate persistent state updates from critical path of function invocations

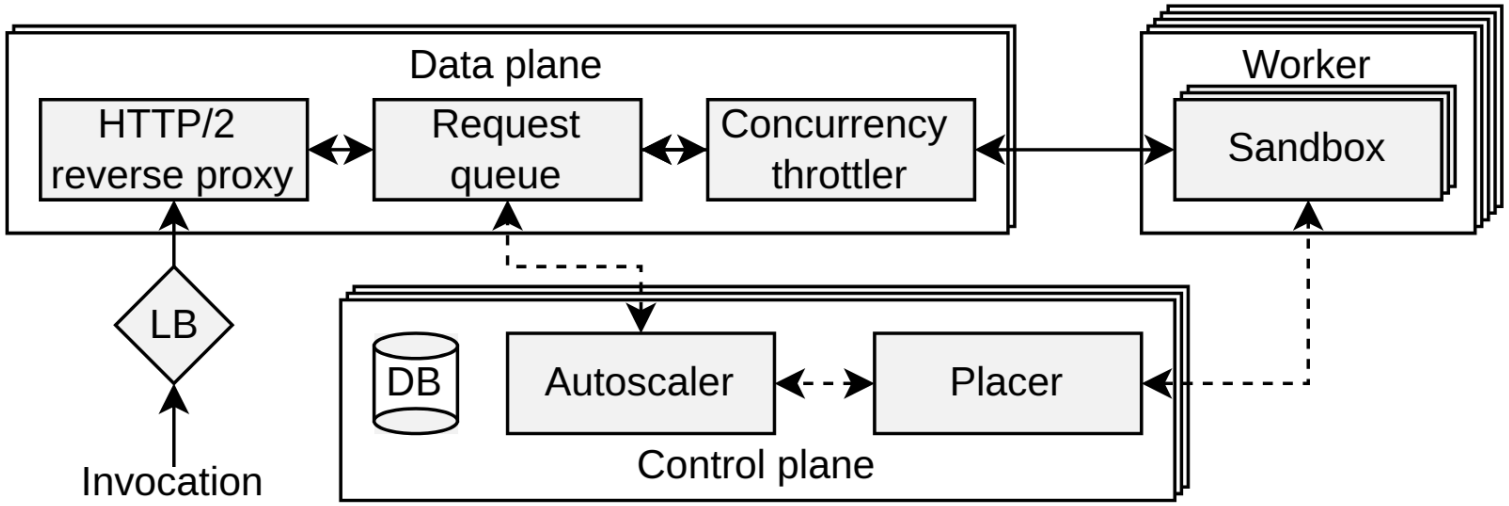
Abstraction	Associated State	Persisted
Function	Name	✓
	Image URL	✓
	Port to expose	✓
	Scheduling configuration	✓
	Scheduling metrics	
Sandbox	Name	
	IP address	
	Port on worker node	
	Worker node ID	
DataPlane	IP address	✓
	Port	✓
WorkerNode	Name	✓
	IP address	✓
	Port	✓

Serverless abstracts servers from users…so if a sandbox fails, don't need to relaunch it on the same server

Dirigent: a new cluster manager for FaaS

Key design principles:

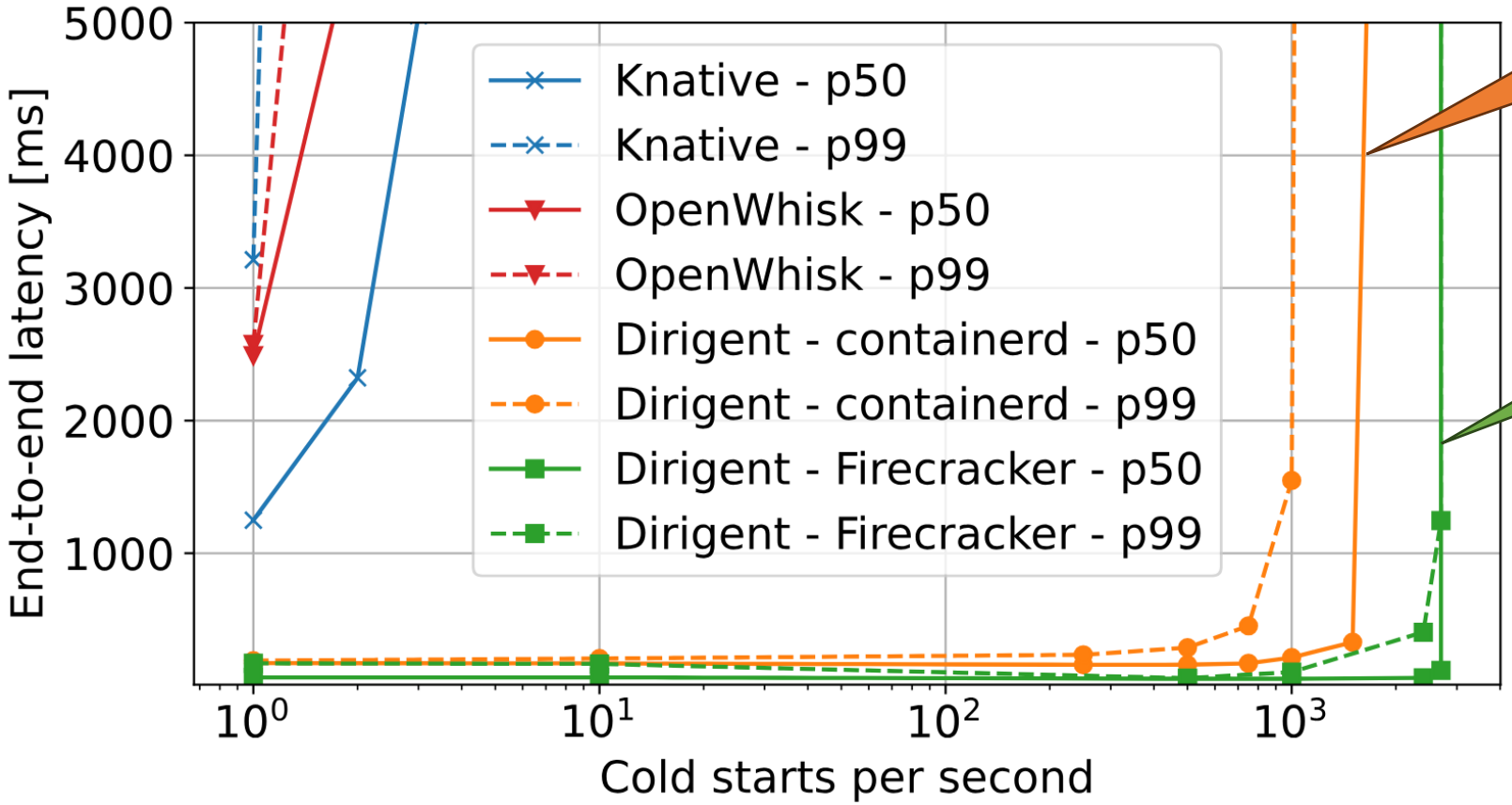
- Simplify cluster management abstractions
- Eliminate persistent state updates from critical path of function invocations
- Monolithic control and data planes



Dirigent evaluation

- 100-node cluster, where 93 nodes are workers
- 10-core xl170 Cloudlab servers with 64 GB DRAM
- Experiment with containerd and Firecracker sandboxes
- Knative v1.13.1 running on top of Kubernetes v1.29.1
- High-availability mode

Dirigent sandbox creation throughput

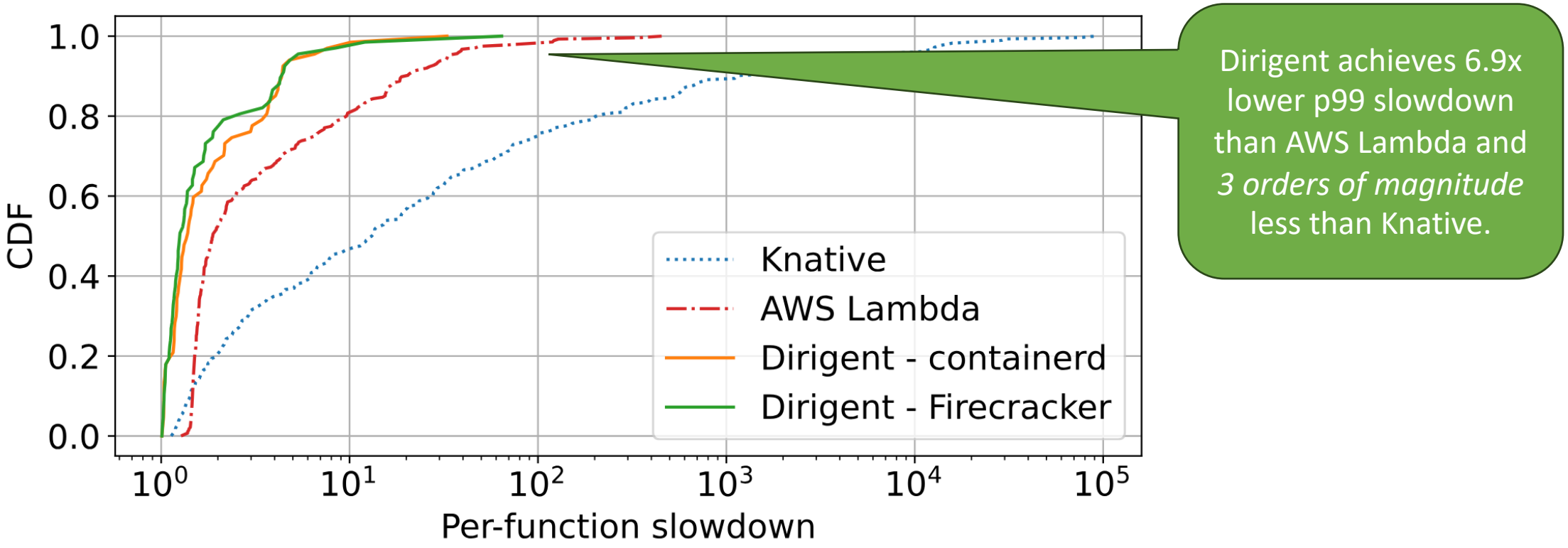


Bottleneck is sandbox creation on containerd workers

Dirigent achieves **1250x** higher sandbox creation throughput than Knative!

End-to-end Latency with Azure Functions Trace

- Azure trace sample with 500 distinct functions in 30min window
- Metric: $slowdown_f = avg\ (e2e_latency_f\ /\ func_exec_time_f)$



Conclusion

- Serverless is a promising new paradigm, however the current system software used to run it is rooted in the old paradigm of managing long-running containers, which limits performance

- Key idea: **rethink cluster manager system architecture and state persistence** to meet serverless application needs

Paper: https://arxiv.org/pdf/2404.16393

Contact: aklimovic@ethz.ch

Website: https://anakli.inf.ethz.ch

ETH zürich

Systems@ETH Zürich

E A S L